



A day in a life of an HANA Administrator

Exciting and challenging insights from real-world
troubleshooting



Matthias Sander
Crossload Consultant
matthias.sander@x-l-c.de

 <https://www.linkedin.com/in/matthias-sander-281240250>

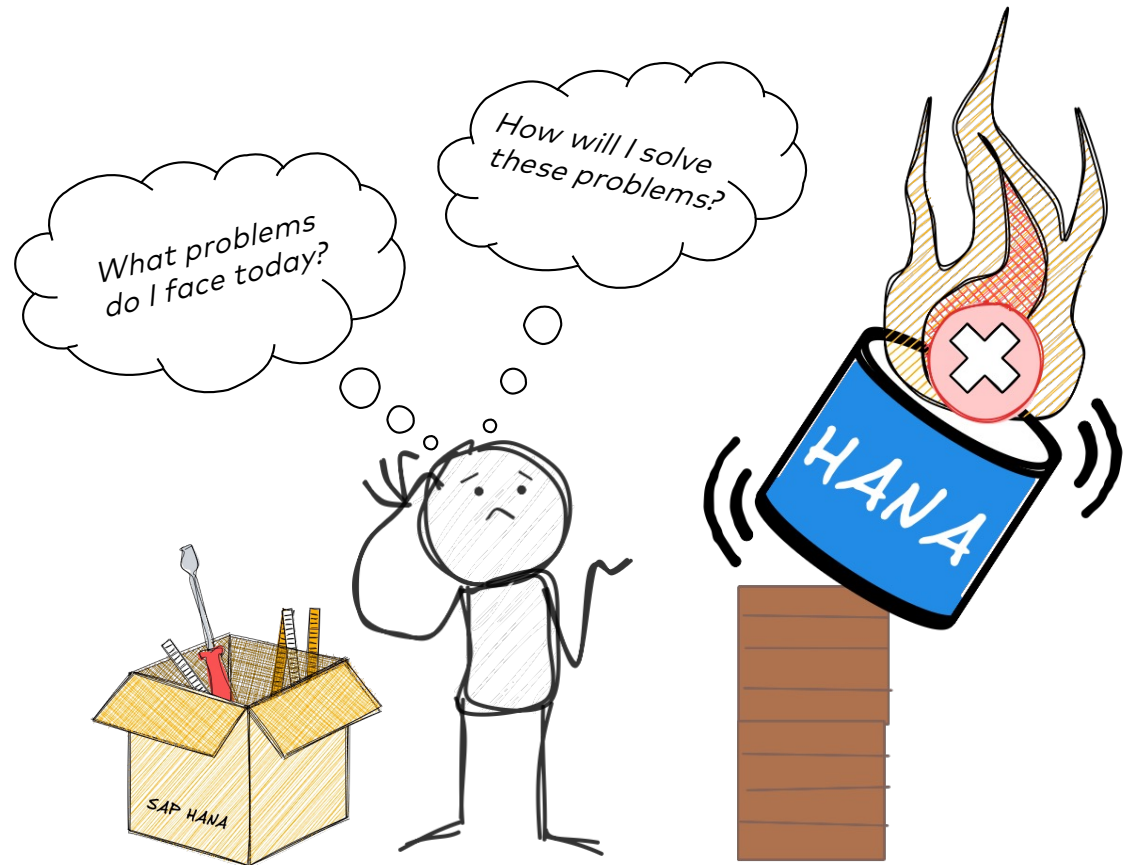
"I have been working in SAP Basis since 2005 and have specialized in SAP HANA for 10 years. As a book author on HANA, I enjoy diving deep into technical details and analyzing complex issues. Honestly, I sometimes enjoy solving tricky HANA problems on a weekend more than watching Netflix."

Agenda:

1. Introduction & Goals
2. Case 1: Trigger & Sequence Cache
 - Lessons Learned & Recommendations
3. Case 2: ABAP Parameters & FDA
 - Lessons Learned & Recommendations
4. Q&A – Only 5 minutes

Note:

If you would like to talk to me after the presentation, please come to the "Expert Corner". I look forward to it.

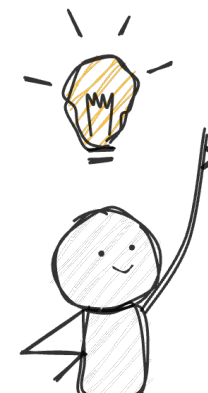


Goals of This Session



What knowledge can I take away?

- Go beyond high-level concepts
- Share real-world troubleshooting examples
- Equip you to analyze and fix performance issues
- Bridge the gap between [developers](#) and [administrators](#)



Starting the day...

➤ 7am: Going to the office with my first coffee



Case 1: Trigger & Sequence Cache

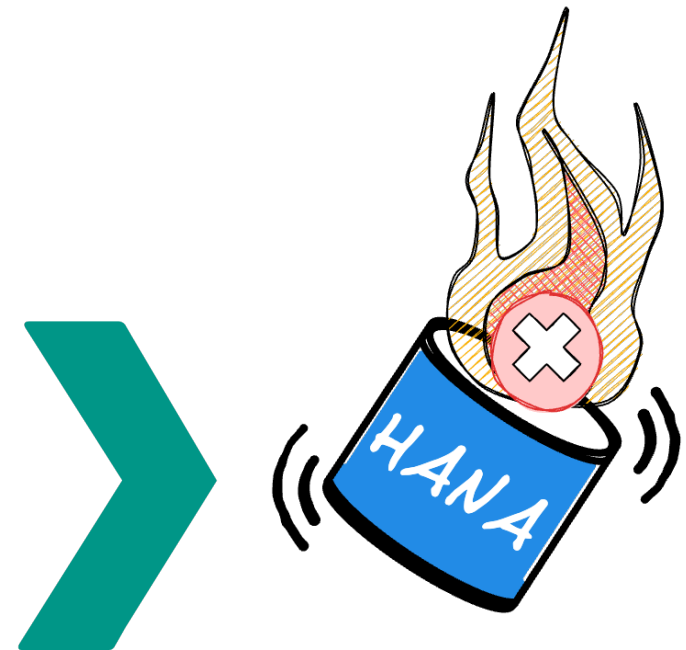
➤ Overview

- The customer has reported a problem when processing a **specific IDOC type**

Problems

- ✗ The process was taking way to long
- ✗ The IDOCS were processed by a background job in ABAP
- ✗ More than one SAP system involved

That was all the information we had at the beginning.



Case 1: Trigger & Sequence Cache

➤ Analyze HANA

- We started to analyze the timeframe when the background jobs were running
 - HANA_Global_TimeFrameReport
- We only found SELECT FOR UPDATE (SU), INSERT (IN) and UPDATE (UP) statements which were expensive
- Which is a little bit **unusual**, more often you will see SELECT statements

STATEMENT_HASH	TP	ELAPSED_S	PER_EXEC_MS	ACT_SESS	EXECUTIONS	RECORDS	REC_PER_EXEC
05...	ob5 SU	5077191	3848.51	176.29	1319259	820190	0.62
66...	UP	963684	178.24	33.46	5406560	4706129	0.87
27...	93 UP	534588	112.83	18.56	4737939	4737939	1
77...	4 IN	514075	108.5	17.85	4737977	4737977	1
6c...	4 UP	432987	91.35	15.03	4739767	4739767	1
3d...	37 IN	411604	86.83	14.29	4740008	4740008	1
c3...	a5 SU	186507	160.41	6.47	1162634	1039027	0.89
5f...	9 UP	121573	297.27	4.22	408963	418912	1.02



Case 1: Trigger & Sequence Cache

➤ Analyze HANA

- In the next step we checked the **specific statements**
 - HANA_SQL_StatementHash_DataCollector
- With the knowledge we had before, we started to look at the "THREAD SAMPLES":
 - „CallStack Not Initialized" - **What's this?**

```
*****  
* THREAD SAMPLES *  
*****  
  
SAMPLES PERCENT HOST   PORT  THREAD_TYPE  
=====
```

19127	100.00	host	30040	SqlExecutor
-------	--------	------	-------	-------------

```
=====
```

SAMPLES	PERCENT	THREAD_TYPE	THREAD_STATE	Q	LOCK_NAME
6165	32.23	SqlExecutor	Semaphore Wait		POSTCOMMIT_FINISH_SMP
5542	28.97	SqlExecutor	Semaphore Wait		Event[:]:@0x7fff5eb390dc: CallStack Not Initialized
4957	25.91	SqlExecutor	Semaphore Wait		Event[:]:@0x7fff5eb2c0a8: CallStack Not Initialized
2391	12.50	SqlExecutor	Running		?
38	0.19	SqlExecutor	ExclusiveLock Enter		BTree GuardContainer

This is a generic, unspecific semaphore name used for various semaphores in IBM on Power environments. Source: [1999998 - FAQ: SAP HANA Lock Analysis](#)



Case 1: Trigger & Sequence Cache

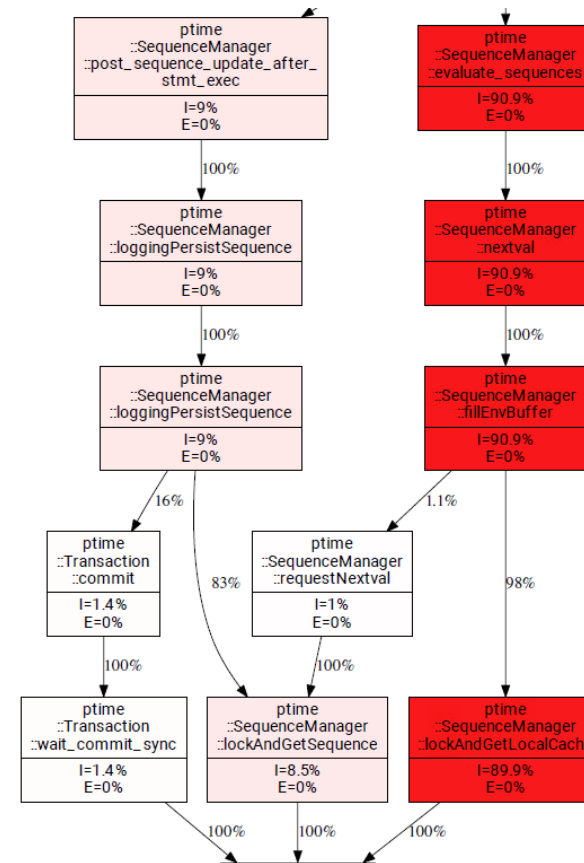
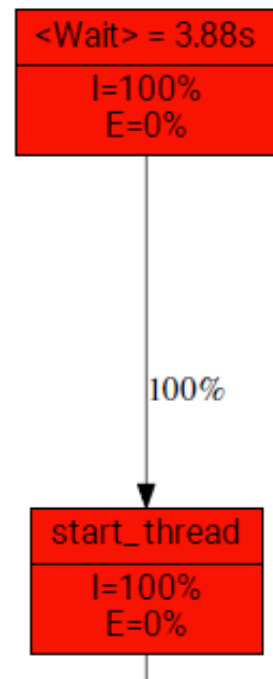
➤ Kernel profiler

- We used the **HANA Kernel profiler** to track down the problem
 - Various call options: Database Explorer, HANA Studio, DBACOCKPIT, SQL (since SPS05), SAP HANASitter and hdbcons
- We found some very interesting moduls:
 - **ptime::SequenceManager::***



Case 1: Trigger & Sequence Cache

Kernel profiler



Case 1: Trigger & Sequence Cache

➤ Kernel profiler

- Based on the information from the kernel profiler, we began investigating **triggers and sequences**
- Also, with this info we had **another look** at the statement data collectors

SAMPLES	PERCENT	CHILD_STATEMENT_HASH	STATEMENT_STRING
251945	89.41	3d38bde70671928bd24d4c4433b69f37	INSERT INTO "SCHEMA"."TABLE" ("TIMESTAMP", "SEQUENCE", "DATA1", "DATA2") VALUES (TO_NUMBER(TO_CHAR(CURRENT_TIMESTAMP, 'YYYYMMDDHH24MISS.FF6')), "SCHEMA"."SEQUENCE".NEXTVAL , \$1, \$2)
25197	8.94	6cfc8904de9e49b8aa483ac43f63fd94	UPDATE "TABLE" SET "DATA1" = "123", "DATA2" = "234" WHERE "MANDT" = ? AND "CUSTOMER" = ?

Case 1: Trigger & Sequence Cache

➤ Kernel profiler

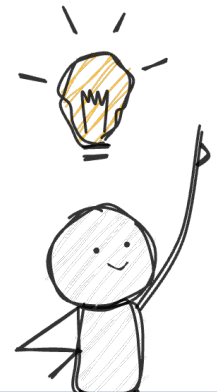
- The affected tables had a **database trigger** to replicate all INSERT/UPDATE/DELETE commands to a third-party tool
- The third party is using a **sequence** in the TRIGGER -> Identified by ".nextval"
- We checked all the used sequences
 - `SELECT * from SEQUENCES where SEQUENCE_NAME like 'Z%'`
- The problem was, that all used sequences were created with **CACHE 1**



Case 1: Trigger & Sequence Cache

➤ Kernel profiler

- Problem as with the ABAP number range without buffering
- If only one data record is changed, `.nextval` directly returns a value
- However, if 1000 records are changed per second, the SQL CMD must wait for the `.nextval` function



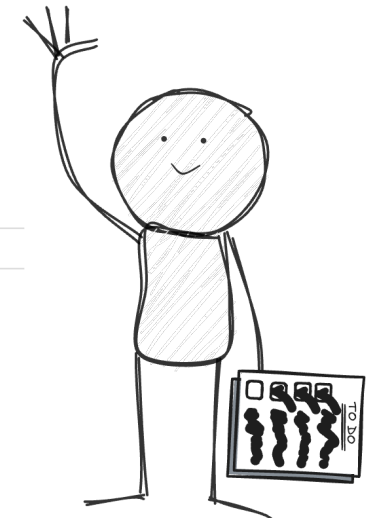
Case 1: Trigger & Sequence Cache

➤ Solution

- Changing the cache:
 - ALTER SEQUENCE "<NAME>" CACHE 100 (or e. g. 1000)
 - [SAP Help ALTER SEQUENCE Statement \(Data Definition\)](#)

Issue	Impacted Revisions	SAP Note	Details
Performance problems for sequence without cache		1999998	If a sequence is defined without a cache, serialization effects and performance problems are possible in case of many concurrent requests, e.g. wait for the "Sequence Manager" mutex. Consider using a cache of e.g. 100 ("CACHE 100") to minimize contention. A cache can result in gaps in the numbers, but this is acceptable for standard SAP scenarios like DDLOG buffer synchronization or SLT replication.

Source: [2600095 - FAQ: SAP HANA Sequences](#)



Case 1: Trigger & Sequence Cache

Before and after

Before:

SNAPSHOT_TIME	HOST	PLAN_ID	ENG	EXECUTIONS	RECORDS	REC_PER_EXEC	ELAPSED_MS	ELA_PER_EXEC_MS	ELA_PER_REC_MS
2023/07/23 08:17:22	hostname	17691330004	C	5078	5078	1.00	31095	6.12	6.12
2023/07/23 07:17:16	hostname	17691330004	C	8464	8464	1.00	41765	4.93	4.93
2023/07/23 06:17:18	hostname	17691330004	C	5213	5213	1.00	30779	5.90	5.90
2023/07/23 05:17:16	hostname	17691330004	C	3507	3507	1.00	21340	6.08	6.08
2023/07/23 03:17:35	hostname	17691330004	C	4953	4953	1.00	31622	6.38	6.38
2023/07/23 01:17:55	hostname	17691330004	C	16472	16472	1.00	203650	12.36	12.36
2023/07/22 23:18:04	hostname	17691330004	C	268515	268515	1.00	4078628	15.18	15.18
2023/07/22 22:18:49	hostname	17691330004	C	651960	651960	1.00	24810757	38.05	38.05
2023/07/22 21:17:48	hostname	17691330004	C	705017	705017	1.00	22550996	31.98	31.98
2023/07/22 20:17:55	hostname	17691330004	C	739665	739665	1.00	43620833	58.97	58.97
2023/07/22 19:17:37	hostname	17691330004	C	1016320	1016320	1.00	98793388	97.20	97.20
2023/07/22 18:17:38	hostname	17691330004	C	977215	977215	1.00	127908093	130.89	130.89
2023/07/22 17:17:42	hostname	17691330004	C	310887	310887	1.00	17328890	55.74	55.74
2023/07/22 16:17:35	hostname	17691330004	C	51633	51633	1.00	416261	8.06	8.06
2023/07/22 15:17:43	hostname	17691330004	C	54506	54506	1.00	416240	7.63	7.63
2023/07/22 14:17:44	hostname	17691330004	C	49864	49864	1.00	352186	7.06	7.06

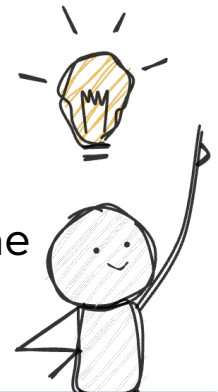
After:

SNP_TIME	HOST	PLAN_ID	ENG	EXECUTIONS	RECORDS	REC_PER_EXEC	ELAPSED_MS	ELA_PER_EXEC_MS	ELA_PER_REC_MS
2023/08/15	hostname	75664130004	C	9835091	9835091	1.00	4960781	0.50	0.50
2023/08/14	hostname	75664130004	C	13354695	13354695	1.00	7244336	0.54	0.54
2023/08/13	hostname	75664130004	C	5532176	5532176	1.00	1770312	0.32	0.32
2023/08/12	hostname	75664130004	C	10441117	10441117	1.00	4661118	0.44	0.44
2023/08/11	hostname	75664130004	C	13885425	13885425	1.00	8524826	0.61	0.61
2023/08/10	hostname	75664130004	C	5621686	1534635291	272.98	3908538	0.69	0.00
2023/08/10	hostname	17691340004	C	5859764	5859764	1.00	44126452	7.53	7.53
2023/08/09	hostname	17691340004	C	6702485	6702485	1.00	76096131	11.35	11.35
2023/08/08	hostname	17691340004	C	6892349	6892349	1.00	79143019	11.48	11.48
2023/08/07	hostname	17691340004	C	5553994	5553994	1.00	58135083	10.46	10.46
2023/08/06	hostname	17691340004	C	2505481	2505481	1.00	23159310	9.24	9.24
2023/08/05	hostname	17691340004	C	6666814	6666814	1.00	61183118	9.17	9.17

Case 1: Trigger & Sequence Cache

» Lessons Learned & Recommendations

- Over 95% of time spent waiting for sequence lock
- The script "HANA_Configuration_MiniChecks" also shows an entry, if caches are set to 1
 - ID1940 - Sequences with insufficient caching
- Always check your DML statements for triggers and if a trigger is used, check the sequences
- Trade-off: Number gaps may appear
 - But SAP recommends caching for high-volume use (Note 2600095)
 - You have to check this behavior for your own development or with the third-party vendor -> In our case the vendor changed the default to cache 100



Case 1: Trigger & Sequence Cache



Relevant SAP notes and links

- [1999998 - FAQ: SAP HANA Lock Analysis](#)
- [2600095 - FAQ: SAP HANA Sequences](#)
- [1969700 - SQL Statement Collection for SAP HANA](#)
- [2800030 - FAQ: SAP HANA Kernel Profiler](#)
- [1804811 - SAP HANA Database: Kernel Profiler Trace](#)
- [Kernel Profiler](#)
- [ALTER SYSTEM {START | STOP | SAVE | CLEAR} KERNEL PROFILER Statement \(System Management\)](#)
- [Graphviz](#)
- [Supportability Tools for SAP HANA](#)

Lunchtime

➤ 11:45am: Pasta and hot coffee



Case 2: ABAP Parameter & FDA

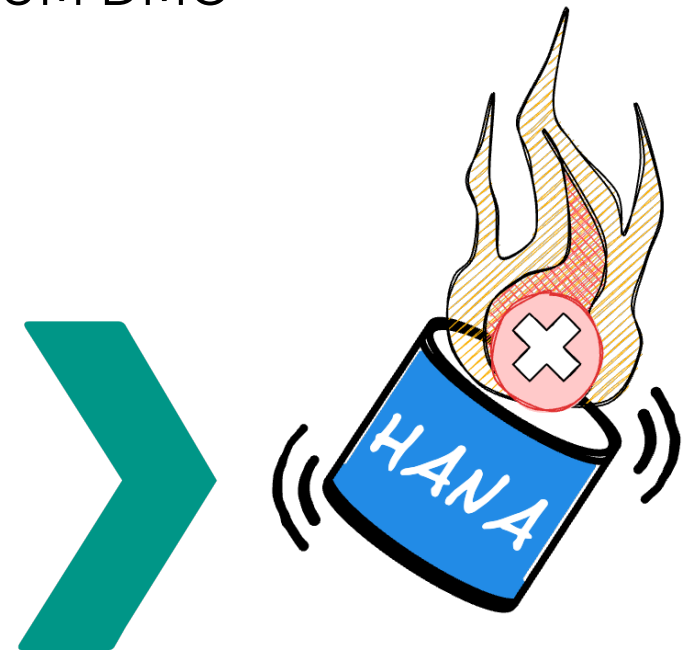
➤ Overview

- Customer migrated from DB2 to HANA via SUM DMO

Problems

- ✗ Specific transaction runtime increased from <2s to >3min
- ✗ No more information available

What could be the reason for this massive difference?



Case 2: ABAP Parameter & FDA

➤ Analyze HANA

- Check HANA parameters according to SAP note 2600030 - Parameter Recommendations in SAP HANA Environments" using the script "HANA_Configuration_Parameters_2.00.040+" from SAP Note 1969700
- Check HANA logs und configuration, e. g. NUMA nodes
- Check HANA load, CPU and memory, for the timeframe when the error was reported

No issues found!



Case 2: ABAP Parameter & FDA

➤ Analyze ABAP

- Trace the problem with ST12
- Check out the summary in the header:
 - Which part is dominant? ABAP or Database?
- In our case the database time was dominant
 - Sort the list "Net (%)" and Type "DB->"
 - Click on "DB->" to find out the SQL statement

Trace analysis - ABAP Trace Per Call

Comment: [redacted] Traced user: [redacted]

ABAP	325,863,142	= 21.9 %
Database	1,164,262,502	= 78.1 %
System	3,185	= 0.0 %
Total	1,490,128,829	= 100.0 %

Execution times in microseconds

Transaction		ABAP	48,071,207	= 19.6 %
Traced user		Database	197,649,273	= 80.4 %
		System	29,366	= 0.0 %
Execution times in microseconds		Total	245,749,846	= 100.0 %

Call	Number	Gross	=	Net	Gross	Net	Progra	Type
Select	116,598	114,184,794	= 11.0	46.5	46.5			DB->
SELECT	66,157	42,897,079	= 42.0	17.5	17.5			DB->
Select	1	17,142,235	= 17.0	7.0	7.0			DB
ABAP	116,598	8,379,623	= 8.0	3.4	3.4			DB->

- Check also SAP Note [3212479](#)

Case 2: ABAP Parameter & FDA

➤ HANA and ABAP

- With the info from ST12 we found the corresponding SQL statement on the HANA database
 - Caller: SAPLKARS:3361
- Statement begins with "SELECT /* FDA WRITE */" -> Fast Data Access

```
*****
* SQL CACHE *
*****
```

SNAPSHOT_TIME	HOST	PLAN_ID	ENG	EXECUTIONS	RECORDS	REC_PER_EXEC	ELAPSED_MS	ELA_PER_EXEC_MS	ELA_PER_REC_MS	PREPARES	PREPARE_MS	LOCK_MS
CURRENT	xxxxxxxxx	4152160003	CE	6	1265	210.83	1247974	207995.59	986.54	2	1212	0
2024/04/16 11:45:20	xxxxxxxxx	4152160003	CE	1	253	253.00	203069	203068.74	802.64	0	0	0
2024/04/16 10:45:20	xxxxxxxxx	4152160003	CE	3	506	168.66	633853	211284.26	1252.67	0	0	0
2024/04/16 09:45:28	xxxxxxxxx	4152160003	CE	1	253	253.00	205284	205284.15	811.39	0	0	0
2024/04/15 18:45:20	xxxxxxxxx	4152160003	CE	1	253	253.00	205768	205767.88	813.31	2	1212	0
2024/04/10 11:45:21	xxxxxxxxx	3860060003	CE	2	69	34.50	428452	214226.13	6209.45	2	1592	0



Case 2: ABAP Parameter & FDA

➤ HANA and ABAP

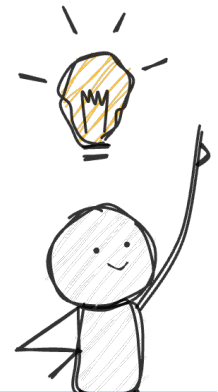
- Use transaction DBA_VIEW_ABAP_SOURCE or program RSDB6GETSOURCE to find the calling ABAP code
 - Yes, it is also possible to find the calling code via ST12
- An FDA statement on the database is triggered by the ABAP function "FOR ALL ENTRIES"
- The decision, if a statement is handled with FDA or in list is made by APAP, not by the database
- The parameter "rsdb/max_in_blocking_factor" controls the behavior on the database



Case 2: ABAP Parameter & FDA

➤ rsdb/max_in_blocking_factor

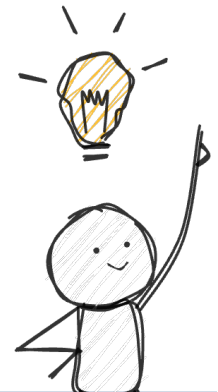
- Itab smaller than or equal to rsdb/max_in_blocking_factor: IN LIST processing
- Itab greater than rsdb/max_in_blocking_factor: IN LIST processing or FDA WRITE processing depending on SAP ABAP kernel level
- The bind values are collected in ABAP in a temporary table **t_00** and transferred to HANA
- When the statement is executed, the values from the temporary table are inserted into the WHERE condition -> "TABLE". "COLUMN" = "t_00". "C_0".
- rsdb/max_in_blocking_factor: The recommendation for a HANA database is 100.



Case 2: ABAP Parameter & FDA

➤ rsdb/max_in_blocking_factor

- As the system was migrated from DB2 to HANA via SUM DMO and the same application server is still used, the parameter rsdb/max_in_blocking_factor was still set to the value 5
- All "FOR ALL ENTRIES" > 5 were already processed using FDA. This can lead to performance issues on the HANA.
- Check also the other ABAP parameters with script HANA_ABAP_MiniChecks



Case 2: ABAP Parameter & FDA

➤ Before and after

- "FOR ALL ENTRIES" has 30 values -> The statement is processed via IN LIST now
- Huge difference for one execution (ELA_PER_EXEC_MS):

EXECUTIONS ▾	RECORDS ▾	REC_PER_EXEC ▾	ELAPSED_MS ▾	ELA_PER_EXEC_MS ▾	ELA_PER_REC_MS ▾	EXEC_PER_EXEC_MS ▾
2	506	253.00	697	348.72	1.37	38.43
1	253	253.00	216379	216378.70	855.25	216378.70

Line 1: Values after the change; Line 2: Values before the change

- The resource consumption was lower after the change:

MEMORY_GB ▾	MEM_PER_EXEC_MB ▾	PLAN_MB ▾	THREADS ▾	THR_PER_EXEC ▾
0.08	43.32	2.13	31	15.50
221.24	226557.87	2.48	1202	1202.00

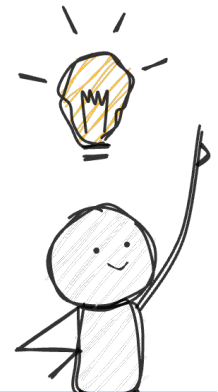
Line 1: Values after the change; Line 2: Values before the change



Case 2: ABAP Parameter & FDA

➤ Lessons Learned & Recommendations

- Always check ABAP parameters after migrating to HANA
 - And after upgrading the SAP system and/or the HANA
 - Use SAP Note [2600030](#) as reference
- FDA behavior controlled in ABAP stack, not DB
- Monitor after updates and SUM/DMO runs
- Collaborate across teams



Case 2: ABAP Parameter & FDA

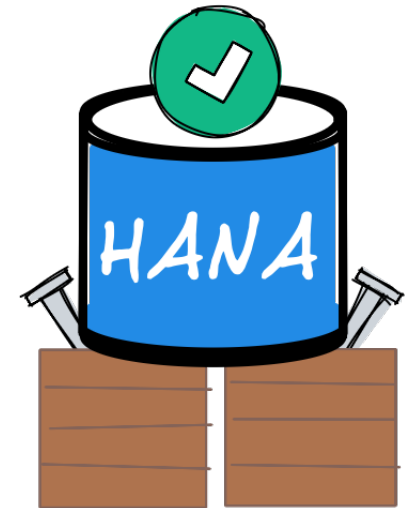
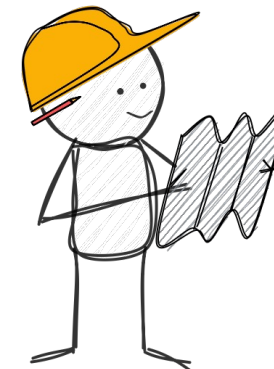
➤ Relevant SAP notes and links

- [2436955 - How to collect and analyze traces using ST12 \(single transaction analysis\)](#)
- [2169881 - How to trace background job using ST12](#)
- [3613302 - How to get the potential bottleneck SQL from a ST12 trace](#)
- [1959493 - How to get ST12 trace for DTP request](#)
- [2524975 - Performance tracing overview: SAT, ST05 & ST12 traces](#)
- [3212479 - How to identify whether it's a HANA Database performance problem according to ST12 trace](#)
- [2600030 - Parameter Recommendations in SAP HANA Environments](#)
- [1969700 - SQL Statement Collection for SAP HANA](#)
- [Single Transaction Analysis](#)

Summary

> A wonderful day...

- At the end, a day as a HANA database administrator could be **very different**
 - Planning and installation new systems
 - Solving tricky problems
 - Data Tiering, like NSE
 - Tuning systems
 - Talking to other teams, like developing
 - And many, many more...
- The database is **running** and the customer is **satisfied**. Then so am I.



End of the day...

➤ 5pm: A little gaming session before playing with my son :-D



More Deep Dive ...

... Examples and additional Information
to this and other Topics can be found in
the first Book written by

Matthias Sander

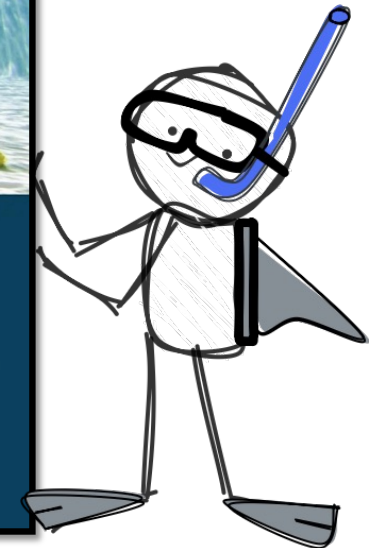
and

Jens Gleichmann

ISBN: 978-3-960124-18-4

(currently only in german)

...but a english Version will follow soon





Thank you for your interest

Contact

Email: matthias.sander@x-l-c.de

LinkedIn: [Matthias Sander](#)

www.x-l-c.de

